

CMPE 492

Automated Game Testing and Difficulty Assessment:
A Structural-Feature Difficulty Oracle for Hex Fall

Alperen AKYOL

Advisor:

Bahri Atay Özgövde

June 10, 2026

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1. Broad Impact	2
1.1.1. Playtest-Free Difficulty Estimation for Live-Ops Pipelines	2
1.1.2. Procedural Generation Along a Target Curve	2
1.1.3. Negative Results as Engineering Guidance	2
1.1.4. Methodological Template for Small-N Applied ML	3
1.2. Ethical Considerations	3
1.2.1. Augmentation vs. Displacement	3
1.2.2. Representativeness of the Winrate Data	3
1.2.3. Responsible Difficulty Engineering	3
1.2.4. Data Confidentiality	4
2. PROJECT DEFINITION AND PLANNING	5
2.1. Project Definition	5
2.1.1. Problem Statement	5
2.1.2. Research Questions (Final)	5
2.1.3. Scope Evolution	6
2.2. Project Planning	6
2.2.1. Timeline as Executed	6
2.2.2. Resources	7
2.2.3. Success Criteria and Outcomes	7
2.2.4. Risk Analysis (Retrospective)	8
2.2.5. Team Work	8
3. RELATED WORK	9
3.1. Curiosity-Driven Reinforcement Learning	9
3.2. Multi-Agent Exploration for State-Space Coverage	9
3.3. Industrial Applications of RL in Game Testing	10
3.4. Diversity-Driven Exploration via Evolutionary Methods	10
3.5. Difficulty Measurement and Procedural Content Generation	10
3.6. Summary and Positioning	11

4. METHODOLOGY	12
4.1. The Game: Hex Fall	12
4.2. Simulator and Level Pipeline	13
4.2.1. High-Throughput Python Simulator	13
4.2.2. Native Level Format Ingestion	13
4.2.3. Gymnasium Environment	14
4.3. Bounded-Rationality Player Ensemble	14
4.4. The Difficulty Oracle	15
4.4.1. Features	15
4.4.2. Model and Validation Protocol	15
4.5. Ablation Discipline and the Solver-Revival Branch	15
4.6. The Original RL Methodology and Why It Was Replaced	17
5. REQUIREMENTS SPECIFICATION	19
5.1. Actors	19
5.2. Functional Requirements	19
5.3. Non-Functional Requirements	20
6. DESIGN	22
6.1. System Architecture	22
6.2. Information Structure	23
6.3. Information Flow	24
6.3.1. Oracle Pipeline	24
6.3.2. Simulation Tick Loop	25
6.3.3. Ablation Decision Flow	26
6.4. Module Decomposition	26
6.5. User Interface	26
7. IMPLEMENTATION AND TESTING	27
7.1. Implementation	27
7.1.1. Simulator and Loader	27
7.1.2. Players and Evaluation	27
7.1.3. Oracle and Ablation Scripts	27
7.1.4. Engineering Discipline	28

7.2. Testing	28
7.3. Deployment	29
8. RESULTS	30
8.1. Headline Result: the Structural Oracle	30
8.2. Why Solvers Carry No Signal	31
8.3. The Solver-Revival Ablations: Four Negatives	32
8.4. Residual Analysis	33
8.5. Summary of Empirical Results	33
9. CONCLUSION	35
9.1. Summary of the Project	35
9.2. On the Value of the Negative Result	35
9.3. Lessons Learned	36
9.4. Future Work	36
REFERENCES	38
APPENDIX A: LEVEL FORMAT AND CORPUS	39
A.1. Native Format Ingestion	39
A.2. Validation	39
A.3. Corpus Characteristics (Aggregate)	39
APPENDIX B: FINAL ORACLE SPECIFICATION	40
B.1. Features	40
B.2. Model	40
B.3. Reproduction	40

1. INTRODUCTION

This project investigates difficulty prediction for the mobile puzzle game *Hex Fall* (Paxie Games). The goal is a **difficulty oracle**: a model that predicts the per-level human winrate of a level from its definition alone, without requiring human playtesting. Such an oracle supports procedural level generation along a target difficulty curve: a level designer (or generator) can evaluate a candidate level instantly instead of waiting for live A/B data.

The project began as a curiosity-driven reinforcement learning (RL) framework for automated game testing. Over the term, the scope narrowed to a single game with real human winrate data, and the methodology pivoted, following advisor consultation, from RL agents to a bounded-rationality solver ensemble, and finally to a minimal structural-feature regression model. The central empirical contribution is twofold:

- (i) A 5-feature structural Ridge regression oracle achieving leave-one-out cross-validated (LOO-CV) **Spearman correlation of 0.6422** (Pearson 0.6210) against real human winrates across a 99-level commercial corpus.
- (ii) A systematic, methodologically disciplined demonstration that **four orthogonal candidate signal axes add no predictive power** beyond the structural baseline on this corpus: solver winrates, graded solver outcomes, an LLM-written human-tactics player, and decision-structure (“Clog”) features.

The second result is an honest negative, and we argue it is the more informative of the two: on this corpus, difficulty in *Hex Fall* is dominated by structural complexity (chiefly total slice count), and behavioral signals derived from bounded-rationality play do not ride on top of it.

1.1. Broad Impact

1.1.1. Playtest-Free Difficulty Estimation for Live-Ops Pipelines

Modern mobile games operate as live services with continuous content production. Each new level traditionally requires human playtesting or live A/B exposure to estimate its difficulty, which is slow, expensive, and risks shipping mis-tuned content to real players. A difficulty oracle that predicts winrate from the level file alone collapses this loop: candidate levels can be scored in milliseconds, and only levels near the target difficulty curve need human validation.

1.1.2. Procedural Generation Along a Target Curve

Paxie Games’ stated need, the framing that shaped this project, is procedural level generation along a target winrate curve. A predictive oracle is the fitness function such a generator requires. Although the perturbation-based generator originally planned for this project was descoped (Section 2.2), the oracle deliverable directly serves this downstream use.

1.1.3. Negative Results as Engineering Guidance

The project’s ablation findings carry practical value beyond the headline model. Knowing that solver-derived behavioral signals do *not* improve winrate prediction on this corpus saves a studio from investing in expensive agent-based evaluation infrastructure where a five-number feature vector suffices. Conversely, the residual analysis (Chapter 8) identifies exactly where the structural model is blind (mechanic interactions such as generator–ice combinations), pointing future effort where it can actually pay off.

1.1.4. Methodological Template for Small-N Applied ML

The corpus contains 99 levels, a small-N regime typical of applied game-analytics problems. The project’s methodology (pre-committed acceptance gates, leakage-free LOO-CV, frame guards that bit-reproduce baselines, hard orthogonality gates, and an explicit no-gate-gaming rule) is a reusable template for drawing defensible conclusions in this regime, where post-hoc tuning can easily manufacture spurious lift.

1.2. Ethical Considerations

1.2.1. Augmentation vs. Displacement

A difficulty oracle automates one narrow, quantitative slice of quality assurance: difficulty estimation. It does not evaluate fun, narrative coherence, visual clarity, or fairness as perceived by players. The tool is positioned as an augmentation that frees human testers and designers for these subjective dimensions, not as a replacement for them.

1.2.2. Representativeness of the Winrate Data

The oracle is fit against aggregate winrates from Paxie’s real player population. Aggregate data hides heterogeneity: a level with 50% winrate may be uniformly moderate or bimodal (trivial for veterans, brutal for newcomers). Predictions inherit whatever population mix produced the training data, and applying the oracle to design decisions for a different player population (e.g., a new market) requires caution. Multi-cohort modeling was explicitly out of scope because the available data is single-aggregate.

1.2.3. Responsible Difficulty Engineering

A tool that predicts difficulty can be used to engineer difficulty, including manipulatively, e.g., inserting deliberate difficulty spikes to drive in-app purchases. The framework provides measurement, not policy; responsibility for how difficulty curves

are shaped rests with the developer. We note that transparent, well-calibrated difficulty progression is also the commercially sustainable choice, as artificial spikes measurably harm retention.

1.2.4. Data Confidentiality

The 99-level corpus and per-level user winrate data are commercially sensitive property of Paxie Games. Throughout the project, the raw data was kept in a git-ignored directory (`CLASSIFIED.paxie_data/`), excluded from version control, and reported only in aggregate form. This report follows the same discipline: no per-level content or per-level winrate values are disclosed.

2. PROJECT DEFINITION AND PLANNING

2.1. Project Definition

2.1.1. Problem Statement

Given a Hex Fall level definition (Paxie’s native editor JSON), predict the fraction of human players who will win the level (`% Win Rate`). The prediction must be derived from the level alone, with no human play data for the target level, so that it can score unseen, procedurally generated levels. Accuracy is measured by rank correlation (Spearman) between predicted and real winrates under leave-one-out cross-validation across Paxie’s 99-level corpus, since the downstream use (ordering candidate levels along a difficulty curve) is fundamentally a ranking task.

2.1.2. Research Questions (Final)

- (i) Can bounded-rationality solver ensembles (greedy, depth- k lookahead, Monte Carlo Tree Search) predict human winrate on a commercial mobile puzzle corpus?
- (ii) Do solver-derived behavioral or decision-structure signals add predictive power beyond structural level features?
- (iii) Can the project’s “Unified Clog Hypothesis” (difficulty as flow obstruction) be operationalized as decision-structure features that are simultaneously orthogonal to level size and predictive of difficulty?

The empirical answers, detailed in Chapter 8, are: (i) no: solver winrate coefficients are near-zero ($|\text{coef}| \leq 0.02$) and a solver-only model achieves *negative* rank correlation (-0.2513); (ii) no: four ablation experiments along orthogonal signal axes all closed between 0.002 and 0.047 Spearman *below* the structural baseline; (iii) no: features passing a hard orthogonality gate against level size were genuinely structurally distinct yet contributed no predictive signal in regression.

2.1.3. Scope Evolution

The project underwent two material scope changes, both documented contemporaneously in the project’s decision log:

- **Mid-April: single-game focus.** The original multi-genre plan (Worm Escape, Color Shooter, Tile Star, Mahjong Infinity) narrowed to *Hex Fall*, the Paxie triple-match puzzle offering the cleanest per-level human winrate data, following industry consultation with Paxie CTO Hüseyin Anıl Özmen.
- **May 29: methodology pivot away from RL.** The curiosity-driven RL methodology was dropped following advisor consultation. Three concerns raised by the advisor motivated the pivot: (a) RL is disproportionate machinery for a sparse-reward puzzle of this size; (b) optimal RL play does not correspond to human play, creating a *structural validity* problem when agent winrate is used to predict human winrate; (c) there is no clean way to simulate skill tiers within RL, since reward tuning changes *what* an agent optimizes rather than *how well* it plays. The replacement, a bounded-rationality solver ensemble plus structural features fit via Ridge regression with LOO-CV, modeled heterogeneous skill explicitly through players of different search depth.

A third, smaller rescope occurred on June 2: after the empirical work closed with the structural oracle and four negative ablations, the advisor judged the results sufficient for the senior design project; the planned perturbation-based level generator was dropped, and all remaining time was allocated to the poster.

2.2. Project Planning

2.2.1. Timeline as Executed

Within Phase 2, the final two weeks (May 30 – June 2) carried the bulk of the empirical work: solver infrastructure, the full evaluation matrix, the oracle fit, four ablation experiments, and the closing residual analysis.

Phase	Duration	Focus	Status
Phase 1	Feb – 29 Mar	Foundation, Clog Hypothesis, portfolio research	Complete
Midterm	29 Mar	Midterm report submission	Complete
Phase 2	30 Mar – 2 Jun	Hex Fall focus, simulator, pivot, empirical work	Complete
Phase 3	3 Jun – 10 Jun	Poster preparation and presentations	Complete

Table 2.1. Term timeline as executed.

2.2.2. Resources

- **Hardware:** Local workstation: Intel i9-13900H (14 cores), RTX 4070 Laptop GPU (8 GB VRAM), 64 GB RAM. No cloud compute was used.
- **Software:** Python 3.11.9; Gymnasium 1.3.0 (environment interface); scikit-learn (Ridge, `StandardScaler`, `cross_val_predict`); scipy, pandas, matplotlib; PyTorch 2.5.1+cu121 and a vendored CleanRL PPO reference (RL pipeline, parked post-pivot).
- **Data:** 99-level Hex Fall corpus with per-level human winrate columns, provided by Paxie Games under confidentiality.

2.2.3. Success Criteria and Outcomes

The acceptance criteria locked at the start of the final push, and their outcomes:

- **Player protocol with ≥ 3 implementations** (greedy, lookahead, MCTS) and passing tests: **met**.
- **Full winrate matrix:** each player $\times$ all 99 levels, ≥ 20 episodes per (player, level) pair: **met** within the time budget (Section 7.2).
- **Oracle fit with a pre-committed gate** of LOO-CV Spearman > 0.5 against % Win Rate: **passed at 0.6422**.
- **Perturbation generator:** descoped per the June 2 advisor decision.
- **Poster for the June 11 session:** **delivered**.

2.2.4. Risk Analysis (Retrospective)

- **Weak oracle correlation (realized in modified form).** The core May 29 hypothesis, that a multi-skill solver mixture predicts human winrate, was *not* supported; the gate still passed because structural features carried the prediction. The planned fallback (feature-only regression) became the headline model. *Mitigation that worked:* the fallback was planned in advance, and the negative was converted into a systematic four-experiment ablation rather than abandoned.
- **MCTS evaluation cost (did not materialize).** The full matrix completed within budget via multiprocessing across 14 cores.
- **Solver variance collapse (realized).** All solvers floored near zero winrate on most levels, eliminating the variance, and therefore the signal, in solver winrates. This is the root cause of all four negative ablations. *Lesson:* signal requires spread; bounded-rationality agents that mostly lose are as uninformative as oracles that always win.
- **Schedule (held).** The 11-day final window had no slack; pre-committed thresholds and a cancel-from-end policy on the experiment queue kept it on track.

2.2.5. Team Work

This project was conducted individually by Alperen Akyol under the supervision of Atay Özgövde, with Paxie Games (CTO Hüseyin Anıl Özmen) as industry partner providing the level corpus and winrate data.

3. RELATED WORK

Research on learning-based agents for automated game testing motivated the project’s original design, and remains relevant background for the pivot decision: understanding why RL-based approaches were ultimately rejected for *this* problem requires understanding what they are good at.

3.1. Curiosity-Driven Reinforcement Learning

Ferdous et al. [1] propose a curiosity-driven RL framework for 3D game testing in which agents are intrinsically rewarded for encountering unfamiliar states, discovering hidden areas and logic errors that scripted approaches miss. This line of work demonstrates that intrinsic motivation improves *exploration coverage*, a bug-finding objective. The present project’s final problem, however, is *difficulty prediction*, where the relevant question is not whether an agent can reach a state but how often a *human* succeeds. This distinction is precisely the structural-validity concern that motivated the May 29 pivot: an agent optimized for coverage or for winning tells us little about human winrate unless its play distribution resembles human play.

3.2. Multi-Agent Exploration for State-Space Coverage

Ferdous et al. [2] extend their earlier framework with coordinated multi-agent exploration, reducing redundancy between parallel agents and accelerating the discovery of multi-step issues. The coordination machinery influenced this project’s original three-layer architecture; it was retired along with the RL methodology, since the final evaluation layer achieves parallelism trivially (independent player-level episodes distributed across cores).

3.3. Industrial Applications of RL in Game Testing

Bergdahl et al. [3] integrate deep RL into testing pipelines at Electronic Arts, emphasizing interpretability and developer-facing artifacts over raw agent performance. Two of their lessons survive into this project’s final form: (a) the deliverable that matters is the artifact a designer can act on (here, a predicted winrate per level file), and (b) agent behavior must be validated against the human behavior it is meant to proxy. Our empirical finding, that bounded-rationality solver winrates carry no signal about human winrates on this corpus, is a concrete instance of (b) failing, detected before the system shipped.

3.4. Diversity-Driven Exploration via Evolutionary Methods

Zheng et al. [4] combine deep RL with evolutionary algorithms (Wuji) to maintain behaviorally diverse agent populations for bug discovery. Behavioral diversity was the inspiration for this project’s multi-skill player ensemble: rather than evolving diversity, we constructed it directly through players of different search depth (greedy, lookahead- k , MCTS), producing three clearly separated skill tiers. The construction succeeded (the tiers were well separated), but the resulting diversity did not translate into predictive signal for human winrate.

3.5. Difficulty Measurement and Procedural Content Generation

Closer to this project’s final problem, Aponte et al. [5] formalize the difficulty of a single-player game as the probability that a player fails a given challenge, and propose estimating it with synthetic players when human data is unavailable. The present project’s target variable, per-level human winrate, is precisely this failure-probability notion measured on a real player population; the contribution here is showing that, for Hex Fall, the quantity can be predicted from level structure directly, and that the synthetic-player route Aponte et al. propose fails on this corpus for a measurable reason (variance collapse, Chapter 8).

On the downstream side, Togelius et al. [6] survey search-based procedural content generation, in which candidate content is generated and selected by an evaluation function. A difficulty oracle is exactly such an evaluation function: the descoped perturbation generator (Section 2.1) would have been a search-based PCG loop with the oracle as fitness. Summerville et al. [7] survey the complementary machine-learning route, where generators are trained on existing content corpora; a corpus-fitted difficulty model like ours pairs naturally with either family, supplying the difficulty axis that generation must be steered along.

3.6. Summary and Positioning

Prior work establishes learning-based agents as effective *explorers* and *bug-finders* in complex environments, formalizes difficulty as failure probability, and treats difficulty evaluation as the fitness function of content generation. The present project asked a different question, whether agent-derived signals predict *human* difficulty on a commercial corpus with real winrate data, and answered it negatively for the bounded-rationality regime, while delivering a simple structural model that does predict it. To our knowledge, systematic per-level human winrate corpora from commercial mobile games are rarely available to academic projects; access to this data, rather than algorithmic novelty, is what allowed the validity question to be tested directly.

4. METHODOLOGY

This chapter details the final methodology: the Hex Fall game and its simulation, the bounded-rationality player ensemble, the structural-feature oracle, and the ablation discipline used to test whether anything improves on the structural baseline. The originally planned curiosity-driven RL methodology is summarized in Section 4.6 for completeness, together with the rationale for its replacement.

4.1. The Game: Hex Fall

Hex Fall is a hexagonal triple-match puzzle with three zones:

- **Hex field (top):** a pile of hexagonal stacks, each composed of colored slices. Only the top slice of each *bottom-row* stack is consumable. When a bottom-row stack is fully cleared, one of its (up to two) upper neighbors falls into the empty position; the choice is uniformly random and is the game’s *only* source of stochasticity.
- **Buffer (middle):** 5 slots holding active buckets. Each bucket has a color and a capacity (24 slices). Buckets in the buffer automatically pull matching top slices from the bottom row; a full bucket leaves the buffer.
- **Bucket reserve (bottom):** a static grid of cells from which the player picks reachable buckets. Cell types include plain buckets, ?-buckets (color hidden until reachable), ice buckets (frozen until a per-bucket move-count threshold), generators (produce a deterministic hidden queue of buckets into their facing cell), walls (permanent obstacles), and pin blockers (rays of blocked cells destroyed when the pin’s destruction cell empties).

The player wins when the hex field is empty and loses when progress becomes structurally impossible (all 5 buffer slots occupied, none can pull, no reserve pick can change this). There is no time or move limit. The game is a partially observable MDP: ?-bucket colors, ice-bucket colors, generator queues, and geometrically occluded

slices are hidden from the player, while the single stochastic element is the fall direction. Full rules and the formal MDP specification are maintained in the project’s `HEXFALL_RULES.md` and `HEXFALL_MDP_SPEC.md` documents.

4.2. Simulator and Level Pipeline

4.2.1. High-Throughput Python Simulator

The game is implemented as a headless Python simulator (`hexfall/game.py`) with no graphical engine, benchmarked at roughly 240 steps/second on a representative commercial level. The per-tick automatic-update loop runs to quiescence after each player action: ice thaw checks → bucket pulls → bucket fill checks → stack-clear falls (the sole RNG consumer) → generator firing → reachability recomputation and ?-bucket revelation → pin destruction checks (with refill protection and intra-tick cascading). Two deterministic tie-break rules (smallest-column-then-row pull targeting; lowest-buffer-slot priority for same-color fill ties) are part of the simulator contract, guaranteeing bit-exact replay from a seed.

4.2.2. Native Level Format Ingestion

The simulator ingests Paxie’s native editor JSON directly, with no translation layer. The loader (`hexfall/level_loader.py`) performs JSON Schema validation plus eight semantic checks (cell exclusivity, coordinate bounds, generator queue consistency, color cross-checks, pin destruction-cell geometry, color token validity, slice–bucket parity at capacity 24, and unsupported-mechanic rejection). Four mechanics present in the wider game but absent from the corpus in meaningful volume (mystery collectors, tied pairs, key locks, hex-tunnels) are hard-rejected: 99 of the 100 corpus levels load; level 100 is rejected on key locks. A critical hygiene rule, locked after a data-integrity audit: all features are computed from the loaded `GameState`, never from the file’s `editorMeta` metadata, which misrepresents color counts on several levels.

4.2.3. Gymnasium Environment

A Gymnasium wrapper (`hexfall/env.py`) exposes visibility-aware observations (the agent sees only what a human would see) and a discrete pick-cell action space with action masking for legality. The wrapper supports `fork()`, cheap state cloning for branch simulation, which the lookahead and MCTS players rely on.

4.3. Bounded-Rationality Player Ensemble

To model heterogeneous human skill without RL, the project implemented a `Player` protocol (`act(obs, env) → action_index`) with four concrete players:

- **GreedyPlayer**: depth-0 heuristic over immediately visible state.
- **LookaheadPlayer**: depth- k search taking the *exact expectation* over stochastic fall outcomes at each branch.
- **MCTSPlayer**: UCT search with greedy rollouts, $N = 100$ rollouts per decision (budget selected by wall-clock spot-check).
- **HumanTacticsPlayer**: a six-component weighted scoring function encoding human play tactics (e.g., buffer-pressure avoidance, pin setup), written via one-shot LLM code generation and running at native speed (introduced in the ablation phase, Section 4.5).

The intended skill spectrum materialized: across the 99-level corpus at 20 episodes per (player, level), aggregate winrates separated into three tiers: lookahead-2 at 0.061, greedy at 0.121, and MCTS at 0.212. (Depth-2 lookahead underperforming greedy is itself a finding: deeper search amplifies a weak leaf evaluator.) The full $3 \times 99 \times 20$ evaluation matrix completed comfortably within the one-hour budget using a per-process environment pool across 14 cores (timing in Section 7.2).

4.4. The Difficulty Oracle

4.4.1. Features

The final oracle uses five structural features extracted from the loaded game state:

- `total_slice_count`: total slices in the hex field (level size);
- `color_count`: number of distinct slice colors;
- `pin_count`: number of pin blockers;
- `ice_count`: number of ice buckets;
- `reserve_area`: size of the bucket reserve grid.

4.4.2. Model and Validation Protocol

The model is Ridge regression with `StandardScaler`, fit against the corpus `% Win Rate` column. Validation is leave-one-out cross-validation via `cross_val_predict` with the scaler *inside* the pipeline, so each held-out prediction is leakage-free: the standardization statistics never see the held-out level. Reported correlations (Spearman, Pearson) are computed between the 99 out-of-fold predictions and the real winrates. The regularization strength and feature set were pre-committed before the fit, together with an acceptance gate of Spearman > 0.5 , to prevent tuning leakage. Spearman is the primary metric because the downstream use, ordering generated levels along a difficulty curve, is a ranking task.

4.5. Ablation Discipline and the Solver-Revival Branch

The initial oracle fit (Chapter 8) revealed that solver winrates contributed nothing: predictive power was entirely structural. Rather than accept this on one experiment, four ablations were queued to test every plausible expansion axis against the structural-only baseline of Spearman 0.6422. Methodological rules were locked *before* the branch began:

- **Pre-committed lift threshold:** an experiment counts as positive only if it lifts LOO-CV Spearman by at least +0.05 over 0.6422; smaller deltas are treated as noise at $N = 99$.
- **Frame guards:** every ablation script first re-fits the structural-only configuration on its joined data frame and must bit-reproduce 0.6422 before any alternative configuration is evaluated, guaranteeing apples-to-apples comparison.
- **Hard orthogonality gate:** candidate decision-structure features must satisfy $|\text{Pearson } r| < 0.7$ against `total_slice_count` or be pruned, preventing re-introduction of level size in disguise.
- **No gate-gaming:** a feature pruned by a gate may not be redefined to slip through it; no post-hoc refits after negative results.
- **Seed reuse:** re-runs of the evaluation matrix reuse the original seeds, so shared columns (e.g., binary winrate) must reproduce bit-identically, a built-in correctness check on the evaluator.

The four experiments, in execution order:

- (i) **Graded solver outcomes (Issue F).** Replace binary win/loss with partial-progress metrics (fraction of slices cleared, moves survived), testing whether solvers’ *partial* progress carries signal even where their winrate floors at zero.
- (ii) **LLM-written human-tactics player (Issue G).** A richer, human-like policy axis: the six-component `HumanTacticsPlayer`, refit into the oracle as an additional winrate feature.
- (iii) **Clog decision-structure features (Issue H).** Operationalizing the project’s Phase 1 “Unified Clog Hypothesis”: three static, search-free metrics, forced-choice count (`fcc`), color transition density (`ctd`), and hidden cell ratio (`rhc`), designed a priori for orthogonality to level size, paper-validated against four hand-picked levels before implementation.
- (iv) **Expanded structural counts (Issue I).** Eight additional structural counts (generator count, wall count, wood-box count, reserve bucket count, generator queue total length, average/maximum field stack height, mechanic diversity),

testing whether the 5-feature set was simply too small.

All four closed negative (results in Chapter 8). The branch design enforced one further discipline: experiments could be cancelled from the end of the queue if budget ran out, but never reordered to chase a promising-looking partial result.

4.6. The Original RL Methodology and Why It Was Replaced

The originally proposed methodology trained PPO agents with an Intrinsic Curiosity Module, modulated by player-capability parameters (perception noise, action precision, exploration temperature) to simulate skill tiers, coordinated as a parallel swarm with shared novelty memory. A complete CleanRL-based PPO training pipeline (vendored reference implementation, action-mask logits masking, TensorBoard logging, wall-clock checkpointing with resume) was built and smoke-verified on hand-built levels before the pivot.

The May 29 pivot replaced this methodology for three reasons raised in advisor consultation:

- (i) **Proportionality.** Hex Fall’s decision problem, while a genuine POMDP, does not require deep representation learning; handcrafted heuristics and shallow search capture the available strategies.
- (ii) **Structural validity.** An agent trained toward optimal play produces a winrate distribution unrelated to the human winrate distribution; using one to predict the other conflates two different random variables. This is not a problem more training fixes; it is a problem with what is being measured.
- (iii) **Skill simulation.** Within RL, reward shaping changes *what* an agent optimizes, not *how well* it plays; there is no principled knob for “play like a mediocre human.” The solver ensemble offers exactly such a knob: search depth.

In retrospect, the empirical results vindicate the pivot’s negative claim (RL agent

winrate would not have predicted human winrate, for the same variance-collapse reason solver winrates did not) while refuting its positive claim (the solver mixture did not carry the prediction either). The evidence ultimately left the structural model standing alone. The PPO pipeline remains in the repository as evidence of the originally planned work and a possible extension path.

5. REQUIREMENTS SPECIFICATION

This chapter states the requirements of the system as finally built: a level-ingestion pipeline, a deterministic simulator, a player-evaluation layer, and a difficulty oracle with a disciplined ablation framework.

5.1. Actors

- **Level Designer / Generator:** supplies level files in Paxie’s native JSON format and consumes predicted winrates and diagnostic reports.
- **Evaluation Pipeline:** the automated actor that loads levels, runs player episodes, extracts features, and fits/applies the oracle.
- **Researcher:** configures experiments, pre-commits thresholds, and interprets ablation outcomes.

5.2. Functional Requirements

- **FR1 (Native format ingestion):** The system shall load Paxie editor JSON directly, performing schema validation and the eight semantic checks defined in the level-format specification, and shall hard-reject levels containing unsupported mechanics with a typed error.
- **FR2 (Deterministic simulation):** Given a level and a seed, the simulator shall reproduce episodes bit-identically, including deterministic tie-breaks for pull targeting and same-color fill collisions; the fall direction shall be the only RNG consumer.
- **FR3 (Full mechanic support):** The simulator shall implement plain buckets, ?-buckets, ice buckets (move-counter thaw), generators (deterministic hidden queues, fire-on-empty-facing-cell), walls, and pin blockers (quiescence-driven destruction with refill protection and intra-tick cascading).
- **FR4 (Visibility-faithful observations):** The environment shall expose to players only information a human player could see, per the game’s geometric

visibility and hidden-information rules.

- **FR5 (Player protocol):** The system shall define a **Player** protocol such that adding a new skill model requires implementing a single class; concrete players shall include greedy, depth- k lookahead, and MCTS.
- **FR6 (Batch evaluation):** The system shall compute a (player $\times$ level) winrate matrix with a configurable episode count, parallelized across CPU cores, with seed reuse for bit-identical replays.
- **FR7 (Feature extraction):** The system shall extract structural features from the loaded **GameState** only, never from **editorMeta** metadata.
- **FR8 (Leakage-free oracle):** The oracle shall be fit via Ridge regression with standardization inside the cross-validation pipeline, reporting LOO-CV Spearman and Pearson against the human winrate target.
- **FR9 (Ablation reproducibility):** Every ablation script shall implement a frame guard that bit-reproduces the structural-only baseline before evaluating alternative configurations.

5.3. Non-Functional Requirements

- **NFR1 (Throughput):** The simulator shall sustain simulation rates sufficient to evaluate the full corpus matrix ($3 \times 99 \times 20$ episodes, MCTS at $N = 100$ rollouts per decision) within one hour on the project workstation. *Achieved (Section 7.2).*
- **NFR2 (Methodological integrity):** Acceptance gates, lift thresholds, and orthogonality gates shall be committed before the corresponding experiments run; gate-rejected features shall not be redefined post hoc.
- **NFR3 (Data confidentiality):** Raw corpus levels and user winrate data shall never enter version control; derived artifacts containing classified data shall be gitignored.
- **NFR4 (Test coverage):** Every mechanic, player, and pipeline component shall be covered by automated tests. *Achieved (Section 7.3).*
- **NFR5 (Extensibility):** Adding a new player or feature shall not require modifying the default oracle; experimental components shall live behind opt-in flags,

leaving the shipped model untouched.

6. DESIGN

The implemented architecture diverges from the originally proposed three-layer RL framework (Learning / Coordination / Analysis). The final system is a four-layer pipeline plus a parked RL layer and a parked decision-structure layer, both retained in the repository for reproducibility.

6.1. System Architecture

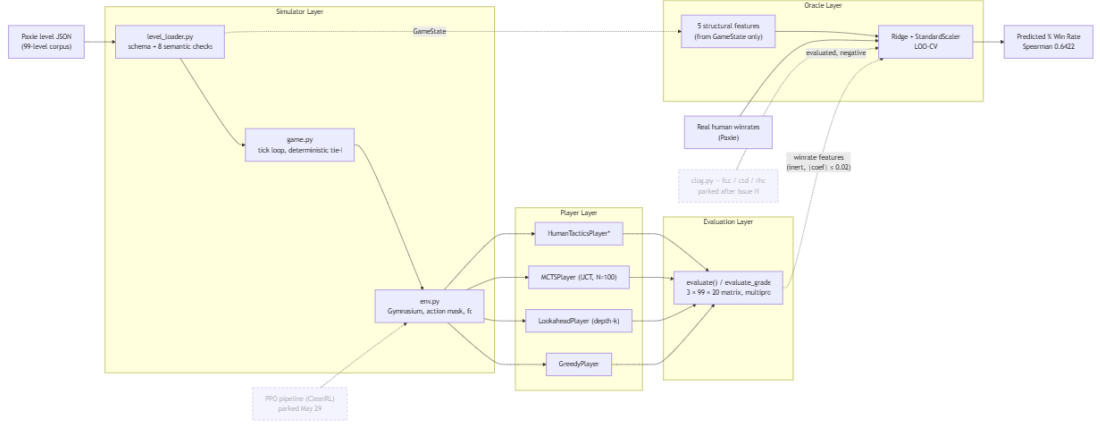


Figure 6.1. Final system architecture. Solid path: the shipped oracle pipeline. Dashed components: built, evaluated, and parked following negative results or the May 29 pivot.

- **Simulator Layer.** Game mechanics (`hexfall/game.py`), level loader (`hexfall/level_loader.py`), typed game-state dataclasses (`hexfall/types.py`), and the Gymnasium wrapper with action masking and state forking (`hexfall/env.py`).
- **Player Layer.** The `Player` protocol and four implementations (greedy, lookahead, MCTS, human-tactics), all deterministic given a seed.
- **Evaluation Layer.** `evaluate()` and `evaluate_graded()` producing per-(player, level) aggregates; multiprocessing pool with a per-process environment instance; seed-reuse contract for bit-identical replays.
- **Oracle Layer.** Feature extractors and the Ridge + `StandardScaler` pipeline (`hexfall/oracle.py`); the fit script producing the predicted-vs-real scatter and correlation statistics (`scripts/fit_oracle.py`).

- **Ablation Infrastructure.** Four sibling fit scripts (`fit_oracle_graded.py`, `fit_oracle_human_tactics.py`, `fit_oracle_clog.py`, `fit_oracle_expanded.py`), each with a frame guard.
- **Decision-Structure Layer (parked).** The three Clog metrics (`hexfall/clog.py`); evaluated in Issue H, not wired into the default oracle.
- **RL Layer (parked).** The CleanRL-based PPO trainer (`scripts/train_ppo.py`) with action-mask logits masking, TensorBoard logging, and wall-clock checkpointing; smoke-verified, parked at the May 29 pivot.

6.2. Information Structure

The simulator’s state model follows the project’s MDP specification:

- **HexField:** hexagonal grid of stack positions (axial coordinates, static topology); per-position ordered slice lists with color labels.
- **Buffer:** 5 slots; per-bucket color, capacity (24), and fill count.
- **Reserve:** square grid (4-adjacency); per-cell content as a tagged union over {empty, plain bucket, ?-bucket, ice bucket, generator, wall}, with optional pin-ray coverage.
- **Pins:** (origin, direction, block count, destroyed flag) tuples; destruction cell derived geometrically.
- **MoveCounter:** first-class state component driving ice thaw.
- **GameState:** the aggregate, from which *all* oracle features are derived (FR7).

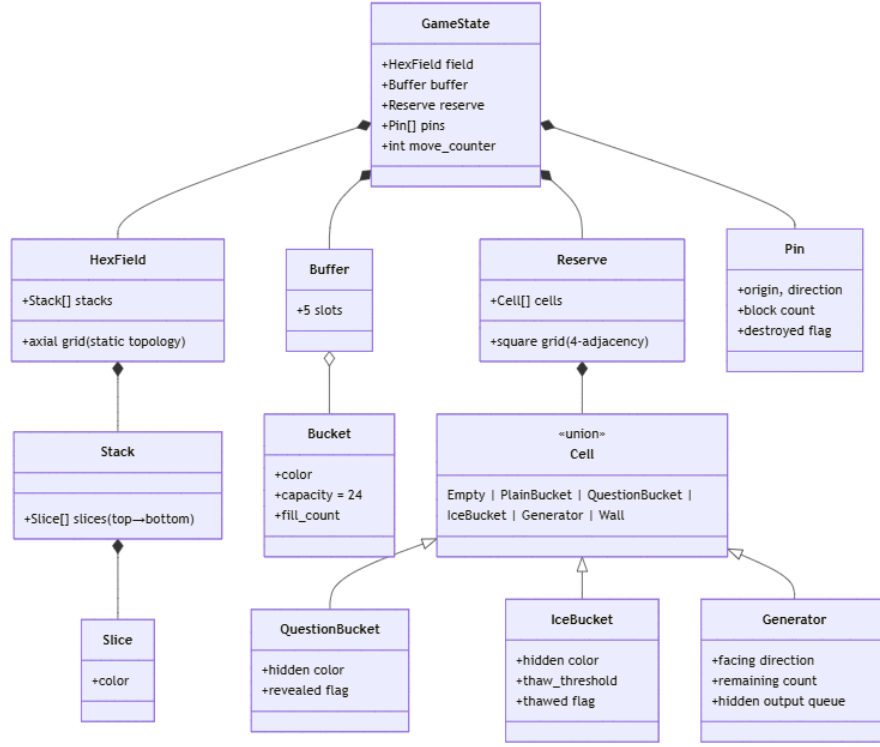


Figure 6.2. Entity model of the Hex Fall game state.

6.3. Information Flow

6.3.1. Oracle Pipeline



Figure 6.3. End-to-end oracle pipeline: level JSON to leakage-free out-of-fold winrate prediction.

A level file flows: loader (validation + semantic checks) → **GameState** → feature extraction (5 structural features) → standardization + Ridge → predicted winrate. During validation, the same flow runs inside a leave-one-out loop, producing one out-of-fold prediction per corpus level; the 99 predictions are correlated against real winrates.

6.3.2. Simulation Tick Loop

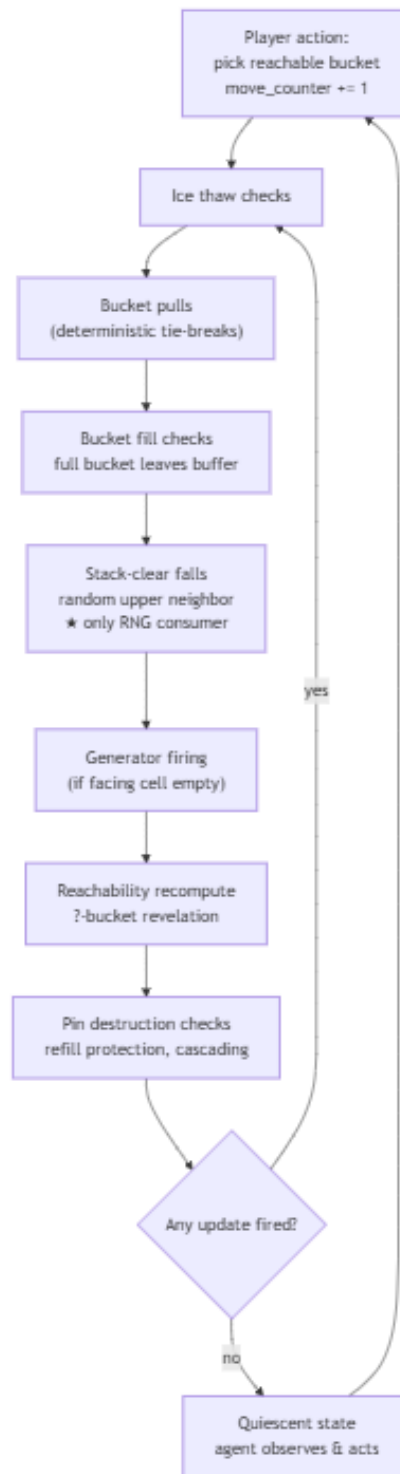


Figure 6.4. Per-tick automatic-update loop, run to quiescence after each player action. The fall step is the single RNG consumer.

6.3.3. Ablation Decision Flow

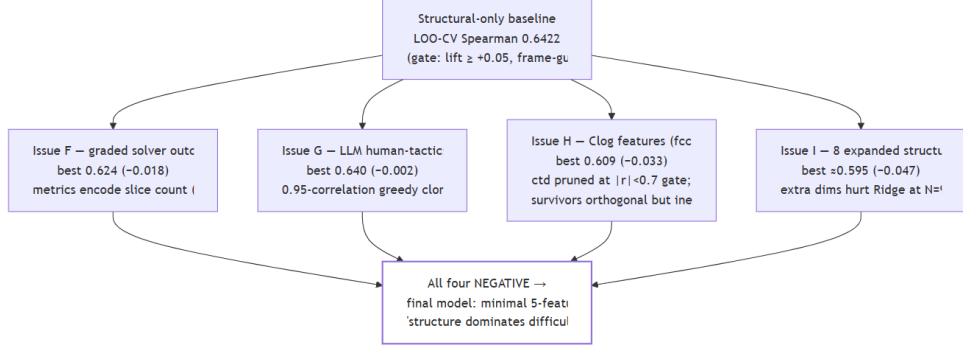


Figure 6.5. Solver-revival ablation branch: four signal axes evaluated against the pre-committed structural baseline; all closed negative.

6.4. Module Decomposition

- `hexfall/`: core package: `types.py`, `level_loader.py`, `game.py`, `env.py`, `players/`, `oracle.py`, `clog.py` (parked).
- `scripts/`: entry points: corpus survey, smoke loaders, evaluation matrices (binary, graded, human-tactics), oracle fits (baseline and four ablations), MCTS budget spot-check, PPO trainer (parked).
- `tests/`: the automated test suite covering loader, mechanics, environment, players, oracle, and Clog features (Section 7.3).
- `levels/`: hand-built test fixtures exercising each mechanic in isolation (generator firing, ice thaw timing, pin cascades, forced deadlocks); the commercial corpus lives outside version control.
- `vendor/`: pinned CleanRL PPO reference.

6.5. User Interface

System is operated through command-line scripts; the designer artifacts are (a) the per-level predicted winrate, (b) the pred-vs-real scatter chart with inline correlation, and (c) the diagnostic residual report grouping mispredictions by mechanic profile. A colorized ANSI renderer for the game state exists for debugging player behavior.

7. IMPLEMENTATION AND TESTING

7.1. Implementation

7.1.1. Simulator and Loader

The simulator implements the complete supported mechanic set: pull/fill/fall/generator phases, reachability recomputation with ?-bucket revelation, ice thaw on the move counter, and pin destruction with refill protection and intra-tick cascading. Implementation surfaced two underspecified situations in the original rules document (multi-stack pull targeting; same-color fill ties); both were resolved with deterministic tie-breaks and flowed back into the specification, keeping document and code in lockstep. The loader rewrite for Paxie’s native format (May 13) eliminated a translation layer entirely: the project ingests the studio’s level files as-is, which also means a future generator can emit directly consumable files.

7.1.2. Players and Evaluation

`GreedyPlayer` and `LookaheadPlayer` shipped first with the `evaluate()` harness and `HexFallEnv.fork()`; `MCTSPPlayer` followed with a rollout budget of $N = 100$ locked by wall-clock spot-check. The full $3 \times 99 \times 20$ evaluation matrix runs in 30.7 minutes via a spawn-based multiprocessing pool. The graded evaluation harness (`evaluate_graded()`) reuses the original seeds; its winrate column reproduced the baseline matrix bit-identically, validating evaluator correctness.

7.1.3. Oracle and Ablation Scripts

`hexfall/oracle.py` implements feature extraction and the Ridge pipeline; `scripts/fit_ora` produces correlations and the headline scatter chart. The four ablation scripts are structured identically: join the candidate features onto the corpus frame, run the frame guard (structural-only must bit-reproduce Spearman 0.6422), then evaluate the

pre-registered configurations. Experimental features live behind opt-in flags (e.g., `include_expanded=False`), so the default oracle is never silently changed by experiment code.

7.1.4. Engineering Discipline

Three practices proved decisive at this corpus size:

- **Audit-first implementation.** Before any change, the implementing session audited specification-vs-code divergences; this caught real bugs across multiple issues, including a metadata field (`editorMeta`) that misrepresents color counts on several levels; this is the origin of the `GameState`-only feature rule.
- **Pre-commitment.** Regularization strength, feature sets, acceptance gates, lift thresholds, and orthogonality gates were fixed before the corresponding experiments ran. At $N = 99$, post-hoc tuning can manufacture several points of spurious Spearman; pre-commitment is what makes the negatives believable.
- **Degenerate-feature vigilance.** The expanded-counts experiment initially specified a `bucket_count` feature that, by the slice-bucket parity invariant, equals `total_slice_count/24` exactly, a perfect affine duplicate. It was caught at design time and replaced with `reserve_bucket_count` (pickable buckets at start, excluding queued generator outputs), restoring genuine variance.

7.2. Testing

The repository closes with **130 passing tests**, grown incrementally ($86 \rightarrow 130$) across the term. Coverage spans:

- **Loader:** schema validation, all eight semantic checks, unsupported-mechanic rejection, color normalization.
- **Mechanics:** per-mechanic fixtures: generator fire-at-load / mid-episode / exhaustion, ?-bucket reveal-on-reachable, ice thaw timing and frozen-cell legality,

pin destruction and single-tick cascades, wall reachability shaping, forced deadlocks, guaranteed-solvable smoke levels.

- **Environment:** observation visibility correctness, action masking, fork semantics.
- **Players:** per-player behavioral tests, including a purpose-built fixture distinguishing the human-tactics player’s pin-setup preference from immediate matching.
- **Oracle and features:** extractor correctness, pipeline leakage checks, Clog metric implementations.

Beyond unit tests, two system-level verification mechanisms ran throughout: seed-reuse bit-identity between sibling evaluation matrices, and frame-guard bit-identity between ablation scripts and the baseline fit.

7.3. Deployment

The system runs locally on the project workstation (Python 3.11.9, virtual environment at the repository root). The locked toolchain: Gymnasium 1.3.0, numpy 1.26.4, scikit-learn/scipy/pandas/matplotlib for the oracle stack, torch 2.5.1+cu121 and TensorBoard 2.20.0 for the parked RL pipeline, CleanRL vendored at a pinned commit rather than installed as a dependency. The full empirical pipeline (corpus load, evaluation matrix, oracle fit, four ablations, residual analysis) is reproducible end-to-end from scripts; no artifact depends on manual steps.

8. RESULTS

All results are computed on the 99 supported levels of Paxie’s 100-level Hex Fall corpus (level 100 is rejected by the loader on an unsupported mechanic) against the corpus’ per-level human % Win Rate, under leakage-free leave-one-out cross-validation.

8.1. Headline Result: the Structural Oracle

Model configuration	LOO-CV Spearman	LOO-CV Pearson
Structural-only (5 features), final oracle	0.6422	0.6210
Structural + 3 solver winrates (8 features)	0.6364	0.6172
Solver winrates only (3 features)	−0.2513	n/a

Table 8.1. Core ablation. Adding solver winrates *degrades* the structural model; solvers alone anti-correlate with human winrate.

The pre-committed acceptance gate (Spearman > 0.5) passed. The headline chart, the predicted-vs-real winrate scatter over 99 out-of-fold predictions, is shown in Figure 8.1.

The standardized coefficients, in order of magnitude:

Feature	Standardized coefficient
total_slice_count	−0.132
color_count	−0.041
reserve_area	+0.038
pin_count	−0.033
ice_count	−0.007

Table 8.2. Final oracle coefficients. Prediction is dominated by level size.

In the joint 8-feature model, all three player-winrate coefficients satisfied $|\text{coef}| \leq 0.02$, an order of magnitude below `total_slice_count`. The May 29 pivot’s positive claim, that a multi-skill solver mixture approximates human winrate, was not supported by the data.

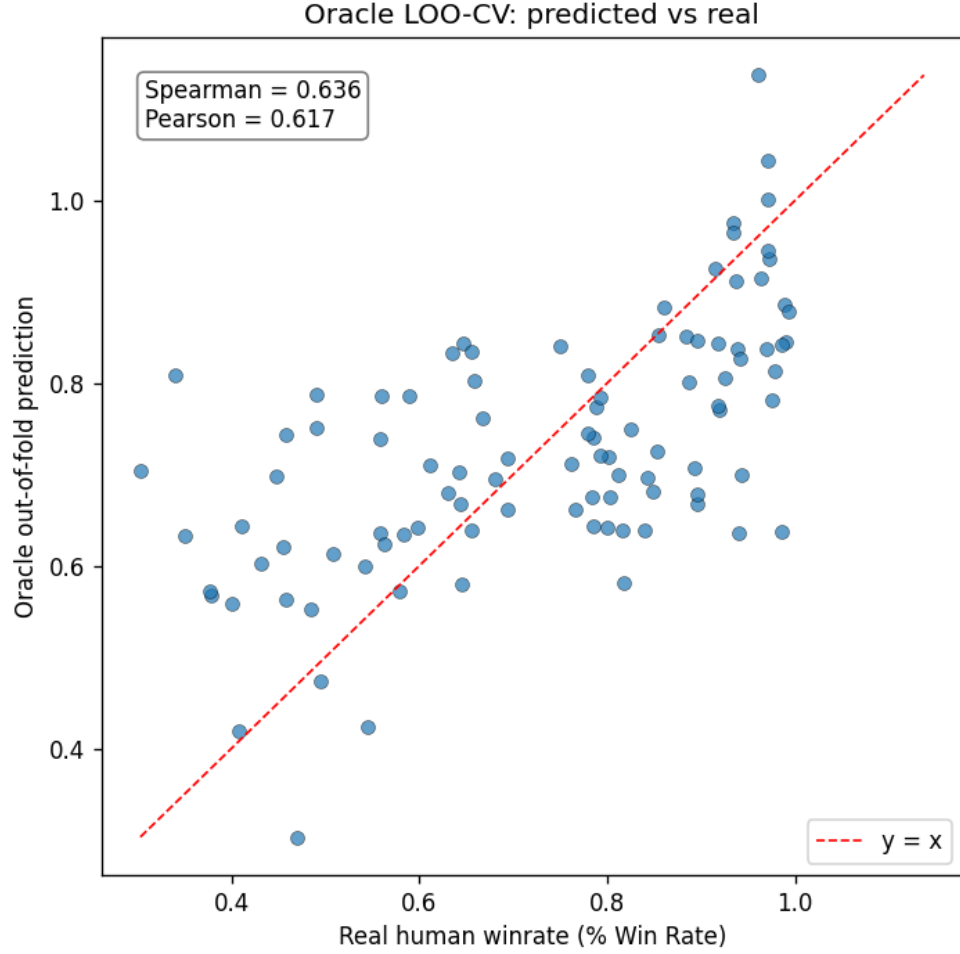


Figure 8.1. Predicted vs. real human winrate, 99 levels, leave-one-out out-of-fold predictions. Spearman 0.6422, Pearson 0.6210.

8.2. Why Solvers Carry No Signal

Diagnosis of the solver failure preceded the ablation branch and shaped it. The bounded-rationality players floor near zero winrate on most commercial levels (aggregate winrates 0.061–0.212, with many per-level zeros), while human winrates span nearly the full $[0, 1]$ range and saturate near 1.0 on easy levels. A feature with no variance carries no signal: where every solver loses, solver winrate cannot distinguish a 40%-human-winrate level from an 85% one. Stronger solvers risk the mirror failure, variance collapse at the ceiling. This variance-collapse mechanism is the root cause behind every negative result that follows.

8.3. The Solver-Revival Ablations: Four Negatives

Issue	Signal axis	Best Spearman	Δ vs. 0.6422
F	Graded solver outcomes	0.624	−0.018
G	LLM human-tactics player	0.640	−0.002
H	Clog decision-structure features	0.609	−0.033
I	Expanded structural counts (8 added)	≈ 0.595	−0.047

Table 8.3. Solver-revival branch: best configuration per experiment. None met the pre-committed +0.05 lift threshold; all closed *below* the baseline.

Per-experiment findings:

- **Issue F (graded outcomes).** Partial-progress metrics (fraction of slices cleared, moves survived) correlate -0.59 to -0.68 with `total_slice_count`: graded solver progress *encodes level size*, duplicating the dominant existing feature rather than adding to it. Graded metrics alone reach only Spearman 0.405.
- **Issue G (human-tactics player).** The six-component player’s winrate is a 0.95-correlation clone of the greedy player’s; under that collinearity, its coefficient sign-flips across configurations, the signature of noise. One caveat is recorded: the player’s speculation component never binds on the real corpus at depth 0, so the negative is scoped as “depth-0-realizable human tactics are insufficient at the oracle level,” not “all human tactics are insufficient.”
- **Issue H (Clog features).** The most informative negative. `ctd` (color transition density) breached the orthogonality gate ($r = 0.728$ vs. slice count) and was pruned without redefinition. The two survivors were *genuinely* orthogonal to level size (`fcc` at $r = 0.022$, `rhc` at $r = 0.451$) and still inert in regression: standardized coefficients $+0.009$ and -0.007 against `total_slice_count`’s -0.117 . Orthogonal but uninformative: on this corpus, decision-structure complexity does not ride on top of size.
- **Issue I (expanded counts).** Eight additional structural counts *degraded* performance (-0.047), confirming that at $N = 99$ extra Ridge dimensions cost more than they contribute and that the minimal 5-feature set is the right size for this

corpus.

8.4. Residual Analysis

A closing diagnostic on the final oracle’s out-of-fold residuals produced three findings. Per the no-refit rule, all are reported as observations; none triggered a parameter update.

- (i) **Win duration as an alternative target.** The same 5 features predict **Avg Win Duration** at LOO-CV Spearman **0.9627** / Pearson 0.9683 (and **Avg Fail Duration** at 0.8704/0.8963). This result carries one prominent caveat: it is partly tautological, since larger levels require more clicks and therefore more time, and it is excluded from the headline for that reason. It is nonetheless practically interesting as a procedural-generation target, since winrate saturates on easy levels (16 of 99 levels have winrate ≥ 0.95) while duration retains resolution across the whole corpus.
- (ii) **Conditional signal in the pruned ctd feature.** `ctd` correlates with the oracle’s residuals at Pearson -0.346 / Spearman -0.328 : holding level size fixed, color mixing within stacks predicts harder-than-expected levels. Marginally redundant with size but conditionally orthogonal; recorded as a methodological observation, not acted upon, since acting on it would bend the pre-committed gate.
- (iii) **Mechanic-specific blind spots.** Grouping levels by mechanic profile reveals systematic mispredictions ($|\text{mean residual}| > 0.05$, $n \geq 5$): ice-only levels are under-predicted (residual $+0.117$, i.e., easier than features suggest), pin-only likewise ($+0.072$); generator-ice combinations are over-predicted (-0.089 ; the combination is harder than the sum of its parts), and wood-box-only mildly over-predicted (-0.053). These are concrete, mechanically interpretable weaknesses of the additive structural model.

8.5. Summary of Empirical Results

Result	Value
Final oracle LOO-CV Spearman vs. % Win Rate	0.6422
Final oracle LOO-CV Pearson vs. % Win Rate	0.6210
Same features vs. Avg Win Duration (secondary target)	0.9627 / 0.9683
Solver-only model Spearman	−0.2513
Structural features in final oracle	5
Supported levels in evaluation corpus	99 of 100
Ablation experiments (all below the +0.05 threshold)	4
Player skill tiers (aggregate winrate: lookahead-2 / greedy / MCTS)	0.061 / 0.121 / 0.212

Table 8.4. Consolidated empirical results.

9. CONCLUSION

9.1. Summary of the Project

This project set out to predict human difficulty for a commercial mobile puzzle game and delivered (a) a complete, tested Hex Fall simulation stack that ingests the studio’s native level format, (b) a bounded-rationality player ensemble with a clean evaluation harness, and (c) a 5-feature structural difficulty oracle at LOO-CV Spearman 0.6422 against real human winrates, validated against four systematic attempts to do better.

The project’s central finding is that **difficulty in Hex Fall, on this 99-level corpus, is dominated by structural complexity, chiefly total slice count**. Behavioral signals from bounded-rationality solvers (binary, graded, or tactically enriched) and decision-structure metrics designed for orthogonality to size add no predictive power. Every plausible expansion axis was tested under pre-committed thresholds and closed negative; the minimal structural model is the model the evidence supports.

9.2. On the Value of the Negative Result

A natural reading of “four experiments failed” is project failure. We argue the opposite. The negatives are informative precisely because they were earned under discipline (pre-registered thresholds, frame guards, hard gates, no post-hoc refits) in a small- N regime where careless analysis manufactures positives easily. For the industry partner, the practical content is direct: a five-number feature vector predicts level difficulty as well as anything built here, including agent simulation that is orders of magnitude more expensive; and the residual analysis says exactly where that simple model is blind (mechanic interactions, intra-stack color structure), so future modeling effort can be aimed rather than scattered.

9.3. Lessons Learned

- **Validity before machinery.** The May 29 pivot, triggered by the observation that optimal agent play is not human play, redirected the project from an approach that would have failed for unmeasurable reasons to one whose failure modes were measurable. The pivot’s own positive hypothesis then also failed, and the measurement framework caught that too.
- **Variance proved to be a prerequisite for signal.** Solver winrates flooring at zero across most of the corpus was the single mechanism behind all four negatives. Any proxy population, whether agents, solvers, or otherwise, must produce outcome *spread* across the items being ranked, or it cannot rank them.
- **Pre-commitment was cheap; credibility was not.** Locking thresholds and gates before the experiments cost nothing and remains the only reason the negative results withstand scrutiny.
- **Specification–code lockstep paid off.** Audit-first implementation and flowing tie-break decisions back into the specification kept a multi-session, multi-tool workflow coherent over four months.

9.4. Future Work

- **Perturbation-based level generation.** The descoped generator remains the natural next step: with a validated oracle as fitness function, perturbation operators that change slice count, the lever the oracle is most sensitive to, can steer generated levels along a target winrate curve.
- **Mechanic-interaction features.** The residual analysis identifies generator–ice combinations and ice-only / pin-only profiles as systematic blind spots; interaction terms or small per-mechanic corrections are the highest-value modeling extension, ideally validated on a fresh corpus to avoid overfitting to the residuals of this one.
- **A larger or multi-cohort corpus.** At $N = 99$, Ridge punishes every added dimension; several negatives (notably the conditionally informative `ctd`) may resolve differently at larger N . Per-cohort winrate data would additionally allow

skill-conditional difficulty prediction, the question the original RL skill-profile design wanted to answer but could not.

- **Duration as a generation target.** Avg Win Duration at Spearman 0.96 retains resolution where winrate saturates; a generator could plausibly target a duration curve where the winrate curve is flat.
- **Human-play traces.** The deepest open question is what bounded-rationality players are missing. Click-level human traces, if ever available, would allow fitting player models to actual human policies rather than guessing at them.

REFERENCES

1. Ferdous, R., F. Kifetew, D. Prandi and A. Susi, “Towards Agent-Based Testing of 3D Games using Reinforcement Learning”, *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*, ACM, Rochester, MI, USA, 2022.
2. Ferdous, R., F. Kifetew, D. Prandi and A. Susi, “Curiosity Driven Multi-agent Reinforcement Learning for 3D Game Testing”, arXiv preprint arXiv:2502.14606, 2025.
3. Bergdahl, J., C. Gordillo, K. Tollmar and L. Gisslén, “Augmenting Automated Game Testing with Deep Reinforcement Learning”, *2020 IEEE Conference on Games (CoG)*, pp. 600–603, IEEE, 2020.
4. Zheng, Y., X. Xie, T. Su, L. Ma, J. Hao, Z. Meng, Y. Liu, R. Shen, Y. Chen and C. Fan, “Wuji: Automatic Online Combat Game Testing Using Evolutionary Deep Reinforcement Learning”, *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 772–784, IEEE, 2019.
5. Aponte, M.-V., G. Levieux and S. Natkin, “Measuring the Level of Difficulty in Single Player Video Games”, *Entertainment Computing*, Vol. 2, No. 4, pp. 205–213, 2011.
6. Togelius, J., G. N. Yannakakis, K. O. Stanley and C. Browne, “Search-Based Procedural Content Generation: A Taxonomy and Survey”, *IEEE Transactions on Computational Intelligence and AI in Games*, Vol. 3, No. 3, pp. 172–186, 2011.
7. Summerville, A., S. Snodgrass, M. Guzdial, C. Holmgård, A. K. Hoover, A. Isaksen, A. Nealen and J. Togelius, “Procedural Content Generation via Machine Learning (PCGML)”, *IEEE Transactions on Games*, Vol. 10, No. 3, pp. 257–270, 2018.

APPENDIX A: LEVEL FORMAT AND CORPUS

A.1. Native Format Ingestion

Levels are ingested in Paxie’s native editor JSON format. The two gameplay-relevant areas are `collectorArea` (the bucket reserve: per-cell typed entries for plain buckets, ?-buckets/wood-boxes, ice collectors, generators/tunnels, dead cells/walls, and pin blockers) and `hexStackArea` (the hex field: per-position slice stacks). The file’s `editorMeta` block is treated as untrusted display metadata (it misstates color counts on several corpus levels) and is never used for feature extraction.

A.2. Validation

On load, the system enforces JSON Schema (draft-07) structural validation plus eight semantic checks: cell exclusivity, in-bounds coordinates, generator queue consistency, color cross-checks between field and reserve, pin destruction-cell geometry, color token validity, slice–bucket parity at bucket capacity 24, and unsupported-mechanic rejection (mystery collectors, tied pairs, key locks, hex-tunnels raise a typed `UnsupportedMechanicError`).

A.3. Corpus Characteristics (Aggregate)

The corpus comprises 100 commercial levels, of which 99 load (level 100 contains key locks). All 99 satisfy slice–bucket parity at capacity 24. Mean total slice counts grow from roughly 459 (levels 1–25) to roughly 806 (levels 76–100), with color counts growing from a mean of 4.2 to 5.0 across the same range, consistent with the corpus implementing a rising difficulty curve through structural growth, which is congruent with the oracle’s finding that structure dominates difficulty. Mechanics are introduced progressively: ice collectors appear from the second quarter of the corpus, pin blockers from the third. Per-level contents and winrates are confidential and not reproduced here.

APPENDIX B: FINAL ORACLE SPECIFICATION

B.1. Features

All features are computed from the loaded `GameState`:

- `total_slice_count`: $\sum_{p \in \text{field}} |\text{stack}(p)|$, the total number of slices across all hex stacks.
- `color_count`: number of distinct slice colors present in the field.
- `pin_count`: number of pin blockers in the reserve.
- `ice_count`: number of ice buckets in the reserve.
- `reserve_area`: number of cells in the reserve grid.

B.2. Model

$$\hat{y} = \mathbf{w}^\top \text{standardize}(\mathbf{x}) + b, \quad \mathbf{w} = \arg \min_{\mathbf{w}} \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2 + \alpha \|\mathbf{w}\|_2^2$$

with α pre-committed before fitting, standardization statistics computed inside each cross-validation fold, and y the per-level human % Win Rate. Validation: leave-one-out cross-validation over the 99 levels via `cross_val_predict`; reported metrics are Spearman and Pearson correlations between the 99 out-of-fold predictions and the real winrates.

B.3. Reproduction

- Fit and headline chart: `python scripts/fit_oracle.py`
- Evaluation matrix: `python scripts/run_eval_matrix.py`
- Ablations: `fit_oracle_graded.py`, `fit_oracle_human_tactics.py`, `fit_oracle_clog.py`,

`fit_oracle_expanded.py` (each begins with a frame guard that must bit-reproduce Spearman 0.6422)

- Tests: `pytest` (130 tests)